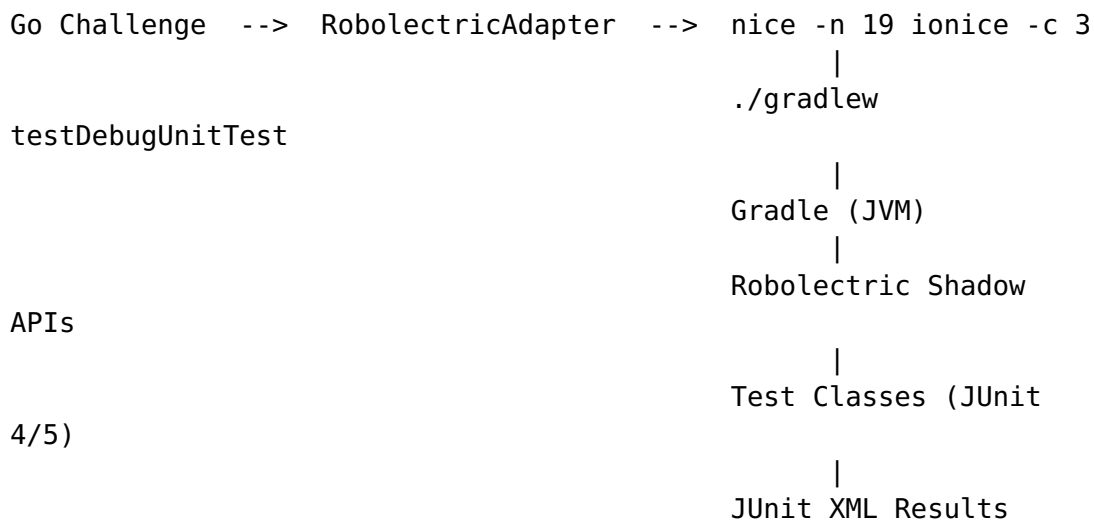


# roboelectric-adapter

## Robolectric Adapter

The Robolectric adapter implements BuildAdapter for running Android unit tests via Robolectric on the JVM. Robolectric simulates the Android framework, allowing Android tests to execute without an emulator or physical device. All commands are delegated to Gradle.

## Architecture



The adapter wraps the Gradle wrapper (./gradlew) and provides three operations: build, test, and lint. All commands are resource-limited with nice -n 19 and ionice -c 3 to comply with the project's resource management rules.

## JUnit XML Parsing

After test execution, the adapter searches standard Robolectric output directories for JUnit XML files:

- build/test-results/testDebugUnitTest/\*.xml
- app/build/test-results/testDebugUnitTest/\*.xml

- `<module>/build/test-results/testDebugUnitTest/*.xml` (multi-module)

These XML files are parsed into `JUnitTestSuite` structs and aggregated into a `TestResult` with counts for total, failed, errored, and skipped tests.

## Prerequisites

1. **Java JDK** installed and in PATH (JDK 11+ recommended)
2. **Gradle wrapper** (gradlew) in the project directory
3. **Robolectric** dependency in the project's build.gradle:

```
testImplementation 'org.robolectric:robolectric:4.12'
```

No emulator or Android device is required.

## Constructor

```
adapter := userflow.NewRobolectricAdapter("/path/to/android-project")
```

## Functional Options

```
type RobolectricOption func(*RobolectricAdapter)
```

| Option                                          | Description                                     | Default                |
|-------------------------------------------------|-------------------------------------------------|------------------------|
| <code>WithRobolectricGradleWrapper(path)</code> | Custom Gradle wrapper path                      | <code>./gradlew</code> |
| <code>WithRobolectricModule(module)</code>      | Gradle module prefix (e.g., <code>:app</code> ) | (root module)          |
| <code>WithRobolectricTestFilter(filter)</code>  | Default test filter for all runs                | (none)                 |
| <code>WithRobolectricJVMArgs(args)</code>       | Additional JVM arguments                        | (none)                 |

## Example with Options

```
adapter := userflow.NewRobolectricAdapter(  
    "/path/to/android-project",  
    userflow.WithRobolectricModule(":app"),  
    userflow.WithRobolectricTestFilter("com.example.unit.*"),  
    userflow.WithRobolectricJVMArgs([]string{"-Xmx2g", "-XX:MaxMetaspaceSize=512m"}),  
)
```

# API Reference

## Build

```
result, err := adapter.Build(ctx, userflow.BuildTarget{
    Name: "debug-apk",
    Task: "assembleDebug",      // Defaults to "assembleDebug" if
                                empty
    Args: []string{"--stacktrace"},
})
```

Executes the Gradle build task. Returns a BuildResult with success status, duration, and output.

## RunTests

```
result, err := adapter.RunTests(ctx, userflow.TestTarget{
    Name:    "unit-tests",
    Task:    "testDebugUnitTest", // Defaults to
                                "testDebugUnitTest" if empty
    Filter:  "com.example.LoginViewModelTest",
})
```

Executes the Gradle test task with optional --tests filter. After execution, searches for JUnit XML results and parses them into structured TestResult data.

The TestResult includes:

```
type TestResult struct {
    Suites      []TestSuite // Parsed from JUnit XML
    TotalTests  int
    TotalFailed int
    TotalErrors int
    TotalSkipped int
    Duration    time.Duration
    Output      string      // Raw Gradle output
}
```

## Lint

```
result, err := adapter.Lint(ctx, userflow.LintTarget{
    Name: "android-lint",
    Task: "lintDebug",      // Defaults to "lintDebug" if
                                empty
})
```

Executes the Gradle lint task and returns a LintResult with the tool name, success status, duration, and output.

## Available

```
ok := adapter.Available(ctx)
```

Returns true when all three conditions are met:

1. The Gradle wrapper file exists at the configured path
2. java is found in PATH
3. `./gradlew --version` executes successfully

## Gradle Integration

### Multi-Module Projects

For multi-module Android projects, the adapter prepends the module prefix to all task names:

```
adapter := userflow.NewRobolectricAdapter(  
    "/project",  
    userflow.WithRobolectricModule(":app"),  
)  
// Tasks  
    become: :app:assembleDebug, :app:testDebugUnitTest, :app:lintDebug
```

### JVM Arguments

JVM arguments are passed via the `-Dorg.gradle.jvmargs` system property:

```
adapter := userflow.NewRobolectricAdapter(  
    "/project",  
    userflow.WithRobolectricJVMArgs([]string{"-Xmx4g"}),  
)  
// Gradle receives: -Dorg.gradle.jvmargs=-Xmx4g
```

### Resource Limits

All Gradle commands are wrapped with resource limiters:

```
nice -n 19 ionice -c 3 ./gradlew <task> [args...]
```

This ensures tests do not consume excessive CPU or I/O, adhering to the project's 30-40% resource limit requirement.

# JUnit XML Parsing

The adapter uses the shared `ParseJUnitXML` function to parse JUnit XML output. It supports both wrapped `<testsuites>` format and single `<testsuite>` format. Parsed results include test case names, class names, durations, failure messages, and stack traces.

Search directories for results (checked in order):

1. `build/test-results/testDebugUnitTest/`
2. `app/build/test-results/testDebugUnitTest/`
3. `<module>/build/test-results/testDebugUnitTest/` (when module is configured)

Each directory is searched for `*.xml` files, with a fallback to one level of subdirectory nesting.

## Limitations

| Aspect       | Detail                                                                          |
|--------------|---------------------------------------------------------------------------------|
| Test scope   | JVM-only tests. Cannot test hardware features (camera, GPS, sensors)            |
| Android APIs | Robolectric shadows cover most but not all Android APIs                         |
| UI testing   | Limited UI testing via Robolectric shadows. For full UI tests, use Espresso     |
| Accuracy     | Robolectric simulates Android; behavior may differ from real devices            |
| Speed        | Faster than on-device tests but slower than pure JUnit (JVM + shadows overhead) |
| No device    | Cannot install APKs, take screenshots, or interact with real devices            |

## When to Use

- Fast feedback during development -- no emulator boot required
- CI/CD pipelines where emulators are impractical
- ViewModel, Repository, and business logic tests
- Robolectric-annotated tests that need Android framework classes

For UI testing or tests requiring a real device, use `EspressoAdapter` or `AppiumAdapter`.

# Integration with Challenge Templates

```
adapter := userflow.NewRobolectricAdapter(  
    "/app/android",  
    userflow.WithRobolectricModule(":app"),  
)  
  
testChallenge := userflow.NewTestRunnerChallenge(  
    "CH-ROBO-001",  
    "Robolectric Unit Tests",  
    "Run Android unit tests via Robolectric",  
    nil,  
    adapter,  
    userflow.TestTarget{  
        Name:    "all-unit-tests",  
        Task:    "testDebugUnitTest",  
    },  
    90.0, // minimum pass rate percentage  
)
```

## Source Files

- Interface: pkg/userflow/adapter\_build.go
- Implementation: pkg/userflow/robolectric\_adapter.go
- JUnit XML types: pkg/userflow/types.go